Student Management System | IT105 – Database Systems

05-03-26

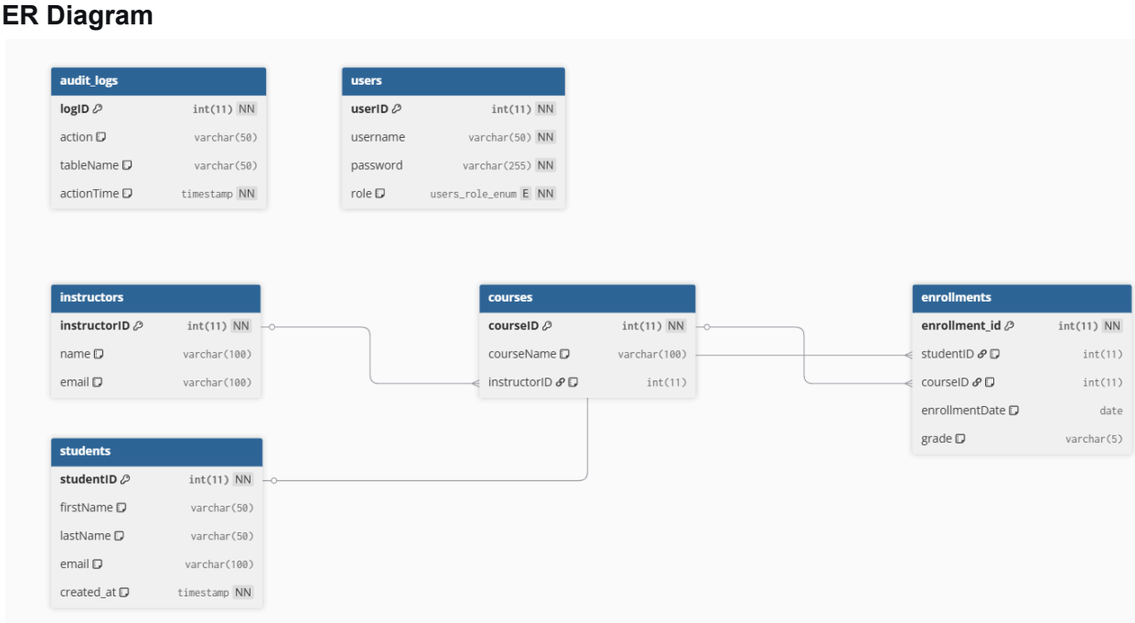
Group Members:
Cantor, John Drex F.
Etorma, Dan Francis S.
Briones, Derick N.
Namia, Kyle Mathew A.
Sambajon, Ryecco Adrian B.
Vinas, Aljon M.
Salcedo, Gabriel Meshach G.

System Description

The Student Management System is a database-driven application designed to efficiently manage student records within an educational institution. It allows administrators, faculty, and staff to store, retrieve, and update student-related information such as personal details, enrollment, grades, and course assignments.

The system aims to replace manual record-keeping processes with a centralized and organized digital solution, improving accuracy, accessibility, and efficiency.

ER Diagram



Database schema and table descriptions

Users Table

Column	Type	Constraints	Description
userID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(50)	UNIQUE, NOT NULL	Login username
password	VARCHAR(255)	NOT NULL	Hashed password
role	ENUM('admin', 'staff')	NOT NULL	User role (admin or

Students Table

Column	Type	Constraints	Description
studentID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique student ID
firstName	VARCHAR(50)		Student's first name
lastName	VARCHAR(50)		Student's last name
email	VARCHAR(100)	UNIQUE	Student's email address
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creating timestamp

Instructors Table

Column	Type	Constraints	Description
instructorID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique instructor ID
name	VARCHAR(100)		Instructor's full name
email	VARCHAR(100)	UNIQUE	Instructor's email address

Courses Table

Column	Type	Constraints	Description
courseID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique course ID
courseName	VARCHAR(100)		Name of the course
instructorID	INT	FOREIGN KEY -> instructors(instructorID)	ID of the instructor teacher this course

Enrollments Table

Column	Type	Constraints	Description
enrollment_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique enrollment record ID
studentID	INT	FOREIGN KEY -> students(studentID)	ID of the enrolled student
courseID	INT	FOREIGN KEY -> courses(courseID)	ID of the course taken
enrollmentDate	DATE		Date when enrollment occured
grade	VARCHAR(5)		Final grade (e.g., A, B+, 85)

Audit_logs Table

Column	Type	Constraints	Description
logID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique log entry ID
action	VARCHAR		Type of action performed (e.g., INSERT, UPDATE, DELETE)
tableName	VARCHAR(50)		Name of the affected table
actionTime	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp when the action occured

Sample SQL queries

Basic Query

```
SHOW TABLES;
DESCRIBE users;
DESCRIBE students;
DESCRIBE instructors;
DESCRIBE courses;
DESCRIBE enrollments;
DESCRIBE a_logs;
```

Screenshots:

Your SQL query has been executed successfully.

```
SHOW TABLES;
```

Profiling [Edit inline] [Edit] [Create PHP code] [Refresh]

Extra options

Tables_in_student_management_systemdb
audit_logs
courses
enrollments
instructors
students
users

Your SQL query has been executed successfully.

```
DESCRIBE users;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
userID	int(11)	NO	PRI	*NULL*	auto_increment
username	varchar(50)	NO	UNI	*NULL*	
password	varchar(255)	NO		*NULL*	
role	enum('admin','staff')	NO		*NULL*	

Your SQL query has been executed successfully.

```
DESCRIBE students;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
studentID	int(11)	NO	PRI	NULL	auto_increment
firstName	varchar(50)	YES		NULL	
lastName	varchar(50)	YES		NULL	
email	varchar(100)	YES	UNI	NULL	
created_at	timestamp	NO		current_timestamp()	

Your SQL query has been executed successfully.

```
DESCRIBE instructors;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
instructorID	int(11)	NO	PRI	NULL	auto_increment
name	varchar(100)	YES		NULL	
email	varchar(100)	YES	UNI	NULL	

Your SQL query has been executed successfully.

```
DESCRIBE courses;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
courseID	int(11)	NO	PRI	NULL	auto_increment
courseName	varchar(100)	YES		NULL	
instructorID	int(11)	YES	MUL	NULL	

Your SQL query has been executed successfully.

```
DESCRIBE enrollments;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
enrollment_id	int(11)	NO	PRI	NULL	auto_increment
studentID	int(11)	YES	MUL	NULL	
courseID	int(11)	YES	MUL	NULL	
enrollmentDate	date	YES		NULL	
grade	varchar(5)	YES		NULL	

Your SQL query has been executed successfully.

```
DESCRIBE audit_logs;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
logID	int(11)	NO	PRI	NULL	auto_increment
action	varchar(50)	YES		NULL	
tableName	varchar(50)	YES		NULL	
actionTime	timestamp	NO		current_timestamp()	

BI Query

```
SELECT
    YEAR(enrollmentDate) AS year,
    MONTH(enrollmentDate) AS month,
    COUNT(*) AS total_enrollments
FROM enrollments
GROUP BY YEAR(enrollmentDate), MONTH(enrollmentDate)
```

Screenshots:

1 16:13:53 SELECT YEAR(enrollmentDate) AS year, MONTH(enrollmentDate) AS month, COUNT(*) AS total_enrollme... 676 row(s) returned 0.313 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	enrollments	NULL	index	idx_enrollment_date	idx_enrollment_date	4	NULL	40352	100.00	Using index; Using temporary

Optimized

```
SELECT
    YEAR(enrollmentDate) AS year,
    MONTH(enrollmentDate) AS month,
    COUNT(*) AS total_enrollments
FROM enrollments
GROUP BY year, month
ORDER BY year, month;
```

Screenshots:

1 19:06:59 SELECT YEAR(enrollmentDate) AS year, MONTH(enrollmentDate) AS month, COUNT(*) AS total_enrollments FROM enrollments GROUP BY year, month ORDER BY year, month LIMIT 0, 1000 676 row(s) returned 0.235 sec / 0.000 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	enrollments	NULL	index	idx_enrollment_date	idx_enrollment_date	4	NULL	40352	100.00	Using index; Using temporary; Using filesort

Role Assignments & Contribution Summary

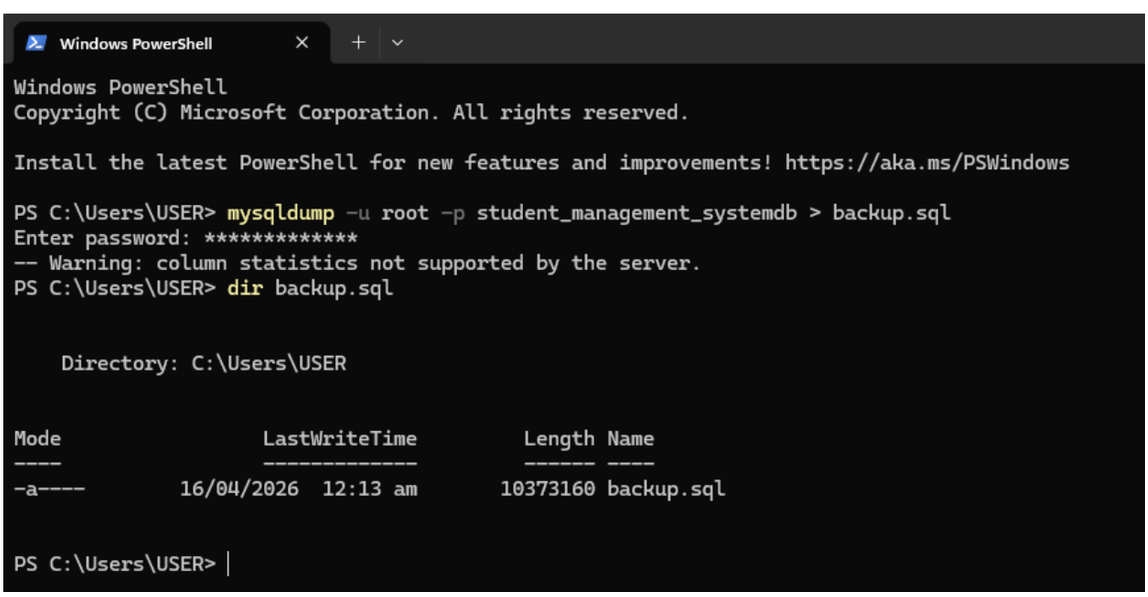
Name	Role	Contribution
Cantor, John Drex F.	Project Manager	Managed the GitHub repository, handled pull requests and merges across multiple project phases, finalized project documentation (README, license), and deployed and maintained the application on cloud infrastructure, ensuring reliability, backups, and seamless system integration.
Etorma, Dan Francis S.	Database Administrator (DBA)	Developed the database schema design and created the star schema for business intelligence, ensuring proper data structure and relationships.
Briones, Derick N.	Documentation Lead	Produced and maintained project documentation including project scope, NoSQL reflection, BI insight report, and emerging technologies reflection, ensuring completeness of all required documents.

Namia, Kyle Mathew A.	SQL Developer	Developed SQL queries for BI analysis, created dataset generation scripts, and prepared CSV datasets for database seeding and testing.
Sambajon, Ryecco Adrian B. Viñas, Aljon M.	QA/Tester	Conducted system testing and validation, identified and reported errors, verified system functionality, and ensured data accuracy and overall system reliability.
Salcedo, Gabriel Meshach G.	Security Officer	Assisted in ensuring data security, reviewed system for vulnerabilities, and supported implementation of safe database and system practices.

Backup and Recovery Steps

Backup Command Used:

```
mysqldump -u root -p student_db > backup.sql
```



Recovery Command Used:

```
mysql -u root -p student_db < backup.sql
```

```
C:\WINDOWS\system32\cmd.

C:\Users\USER>mysql -u root -p student_management_systemdb < backup.sql
Enter password: *************

C:\Users\USER>mysql -u root -p
Enter password: *************
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 234
Server version: 5.5.5-10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+----------------------------+
| Database                   |
+----------------------------+
| backup                     |
| ibu-notice-system          |
| information_schema         |
| mysql                      |
| performance_schema         |
| phpmyadmin                 |
| student_management_systemdb |
| studentsystemdb            |
| test                       |
+----------------------------+
9 rows in set (0.00 sec)

mysql>
```

Indexing & Performance Improvements

```
--  ENROLLMENTS
--
CREATE INDEX idx_enrollments_student_course ON enrollments(studentID,
courseID);
CREATE INDEX idx_enrollments_grade ON enrollments(grade);
CREATE INDEX idx_enrollments_date_grade ON enrollments(enrollmentDate,
grade);

-- STUDENTS
CREATE INDEX idx_students_lastname ON students(lastName);
CREATE INDEX idx_students_firstname ON students(firstName);
CREATE INDEX idx_students_email ON students(email);
CREATE INDEX idx_students_name ON students(lastName, firstName);

--
-- COURSES
--
CREATE INDEX idx_course_instructor ON courses(courseID, instructorID);
CREATE INDEX idx_courses_name ON courses(courseName);

--
-- AUDIT LOGS
--
CREATE INDEX idx_audit_action ON audit_logs(action);
CREATE INDEX idx_audit_tableName ON audit_logs(tableName);
CREATE INDEX idx_audit_actionTime ON audit_logs(actionTime);
CREATE INDEX idx_audit_table_action ON audit_logs(tableName, action);

--
-- DIM TABLES
--
CREATE INDEX idx_dim_student_id ON dim_student(studentID);
CREATE INDEX idx_dim_student_email ON dim_student(email);
CREATE INDEX idx_dim_student_name ON dim_student(lastName, firstName);

CREATE INDEX idx_dim_course_id ON dim_course(courseID);
CREATE INDEX idx_dim_course_name ON dim_course(courseName);
CREATE INDEX idx_dim_course_instructor ON dim_course(instructorID);

CREATE INDEX idx_dim_instructor_id ON dim_instructor(instructorID);
CREATE INDEX idx_dim_instructor_name ON dim_instructor(name);
CREATE INDEX idx_dim_instructor_email ON dim_instructor(email);

CREATE INDEX idx_dim_date_full ON dim_date(fullDate);
```

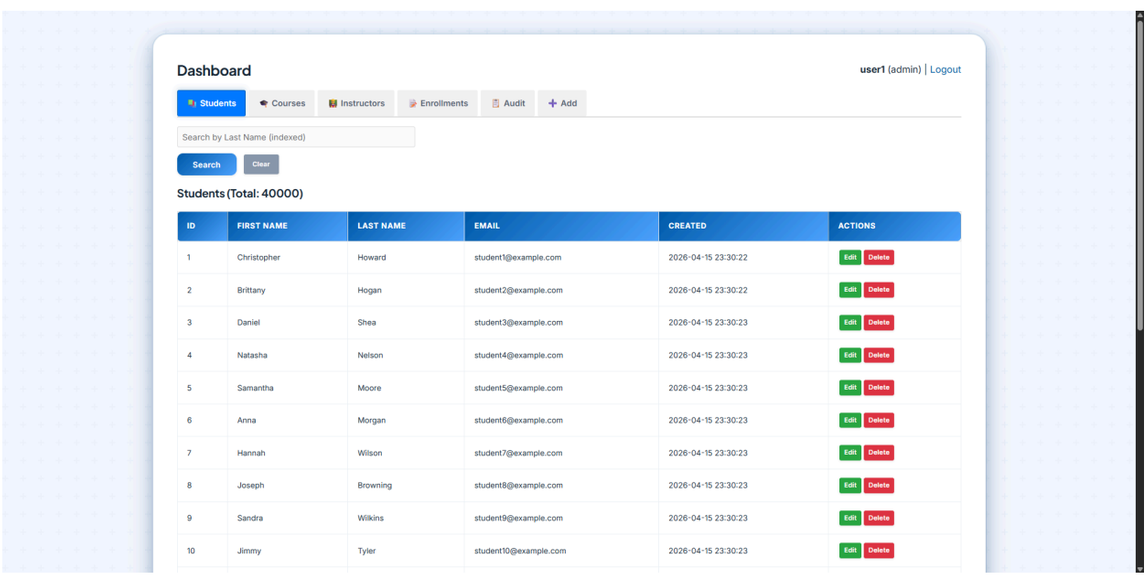
Indexes were created on several tables to improve the speed of queries in the system. For the enrollments table, indexes were added on studentID, courseID, grade, and enrollmentDate to make searching, filtering, and reporting faster.

In the students table, indexes were placed on names and email to make it easier to find specific students. The courses table was also indexed to quickly access course and instructor information.

Audit logs were indexed based on actions, table names, and timestamps to improve tracking and monitoring of system activity. Additionally, indexes were applied to dimension tables to support faster data retrieval for reporting and analysis.

Overall, these indexes help the system run more efficiently by reducing the time needed to access and process data.

DASHBOARD



Reflection and Limitations

The development of the Student Management System allowed the group to apply database concepts such as normalization, indexing, and query optimization in a practical setting. The project improved our understanding of relational database design and strengthened our collaboration as a team.

The system includes a graphical user interface for easier interaction; however, it still has some limitations. For example, identifying students who are not enrolled in any course requires additional queries and is not directly shown in the system. Security features are also basic and need improvement through better authentication and access control. Future enhancements may include improving these features and adding real-time analytics.